



Programación 1

Arreglos

Arreglos

- Estructura de datos que contiene elementos o celdas del mismo tipo de dato.
- Son estáticos ya que su tamaño no cambia durante la ejecución
- Ocupa un serie de posiciones contiguas de memoria
- Cada celda tiene asociado un número entero que indica la posición respecto de las demás (índice)



POSICIONES→

```
int A = vec[0] + vec[8];    // A = 12 + 6 = 18
int B = 2 + vec[3];        // B = 2 + 8 = 10
vec[0] = A + B;            // vec[0] = 18 + 10 = 28
```

Declaración de arreglos

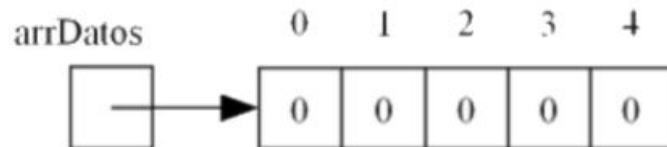
Arreglo nulo sin espacio para datos

```
int[] arrDatos;
```



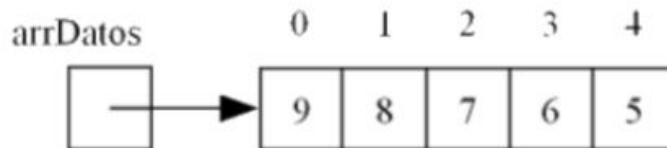
Arreglo inicializado con ceros

```
int[] arrDatos = new int[5];
```



Arreglo inicializado con datos explícitos

```
int[] arrDatos = {9, 8, 7, 6, 5};
```



Acceso a un arreglo



Una vez que el arreglo tiene espacio asignado el paso siguiente es inicializarlo con valores o cargarlo desde el teclado. Para acceder a cada espacio del arreglo se utiliza un valor de posición o variable con dicho valor, que va desde **0** a la **CANTIDAD-1**. Por ejemplo:

```
arrDatos[0] = 1;  
//se accede a la posición 0 del arreglo y se le asigna un 1  
  
pos = 2; //suponiendo que pos es una variable de tipo entero  
arrDatos[pos] = 5;  
//se accede a la posición pos del arreglo y se le asigna un 5, pos tiene que tener un  
valor entre 0 y  
//CANTIDAD-1 de elementos del arreglo, sino dará ERROR
```

El acceso a una posición mediante un valor **no** se recomienda, siempre se utiliza una variable entera cuyo valor va de **0** a la **CANTIDAD-1**.

Mostrar por consola un arreglo



//Hacer un programa que muestre un arreglo por consola que fue definido por extensión.

```
public class Clase_5_Ejemplo_1{
    final static int MAX = 5;
    public static void main (String[] args) {
        int[] enteros = {24, 32, 15, 12, 18};
        mostrarArreglo(enteros);
    }
    public static void mostrarArreglo(int[] arrenteros){
        for (int pos = 0; pos < MAX; pos++){
            System.out.println("arrenteros["+pos+"] -> "+arrenteros[pos]);
        }
    }
}

} // fin del class
```

Obtener el promedio de valores de un arreglo

//Hacer un programa que dado un arreglo de enteros de tamaño 10 que se encuentra
//precargado, imprima por pantalla el promedio de sus valores.

```
public class Clase_5_Promedio {
    public final static int MAX = 10;
    public static void main (String[] args) {
        int [] arrenteros = {55, 84, 12, 69, 14, 22, 4, 7, 100, 2};
        double promedio;
        promedio = promedio_arreglo(arrenteros);
        System.out.println("El promedio del arreglo es: " + promedio);
    }
    public static double promedio_arreglo(int[] arr){
        int suma = 0;
        for (int pos = 0; pos < MAX; pos++){
            suma+=arr[pos];
        }
        return ((double)suma/MAX);
    }
}
```

Obtener la posición de un elemento del arreglo 1/2

Hacer un programa que dado un arreglo de enteros de tamaño 10 que se encuentra precargado, encuentre la posición de un número entero ingresado por el usuario. Si existe, muestre esa posición por pantalla, o indique que no existe.

```
public class Clase_5_Ejemplo_2 {
    public final static int MAX = 10;
    public static void main (String [] args) {
        int [] arrenteros = new int [MAX];
        arrenteros = {55, 84, 12, 69, 14, 22, 4, 7, 100, 2};
        int numero = obtenerNumero();
        int pos = obtener_pos_arreglo(arrenteros,numero);
        if (pos<MAX){
            System.out.println(numero + " está en " + pos);
        }
        else{
            System.out.println("No existe " + numero);
        }
    }
}
```


Obtener la posición de un elemento del arreglo 2/2



```
public static int obtenerNumero(){
    int numero = 0;
    System.out.println("Ingrese un numero entero :");
    numero = Utils.leerInt();
    return numero
}
```

```
public static int obtener_pos_arreglo(int [] arr, int numero){
    int posicion = 0;
    while ((posicion < MAX) && (arr[posicion] != numero)){
        posicion++;
    }
    return posicion;
}
```

Formas de pasar parámetros



Por copia valor:

- Cuando se invoca al método se crea una nueva variable (el parámetro formal) y se le copia el valor del parámetro actual.
- El parámetro actual y el formal son dos variables distintas aunque puedan llegar a tener el mismo nombre.
- El método trabaja con la copia de la variable por lo que cualquier modificación que se realice sobre ella dentro del método no afectará al valor de la variable afuera.

Por referencia:

- Cuando se invoca al método se crea una nueva variable (el parámetro formal) a la que se le asigna la dirección de memoria donde se encuentra el parámetro actual.
- En este caso el método trabaja con la variable original por lo que puede modificar su valor.

Pasaje de parámetros en java



En Java **todos** los parámetros se pasan por **copia valor**.

- Cuando el argumento es de tipo **primitivo**, el paso por copia valor significa que cuando se invoca al método se reserva un nuevo espacio en memoria para el parámetro formal. El método no puede modificar el parámetro actual.
- Cuando el argumento es una **referencia** (por ejemplo, un array o cualquier otro objeto) el paso por copia valor significa que el método recibe una copia de la dirección de memoria donde se encuentra el objeto. Por lo tanto el método puede modificar el contenido.

Ejemplo

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] B = new int[MAX];  
        int a = 20;  
        cargar_arreglo(B);  
        System.out.println("Los datos son:");  
        for (int pos = 0 ; pos < MAX; pos++)  
            System.out.println(B[pos]);  
        cargar_variable_simple(a);  
        System.out.println("La variable es:"+a);  
    }  
    private static void cargar_variable_simple(int c) {  
        c = 10;  
    }  
    private static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
}
```

Memoria RAM - main

B = |0|0|0|0|0|
a = 20

Ejemplo

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] B = new int[MAX];  
        int a = 20;  
        cargar_arreglo(B);  
        System.out.println("Los datos son:");  
        for (int pos = 0 ; pos < MAX; pos++)  
            System.out.println(B[pos]);  
        cargar_variable_simple(a);  
        System.out.println("La variable es:"+a);  
    }  
    private static void cargar_variable_simple(int c) {  
        c = 10;  
    }  
    private static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
}
```

Memoria RAM - main

B = |0|0|0|0|0|
a = 20

Memoria RAM -
cargar_arreglo

arr = 0xf3a3fd4a
(recibe la dirección de
memoria de B)

Ejemplo

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] B = new int[MAX];  
        int a = 20;  
        cargar_arreglo(B);  
        System.out.println("Los datos son:");  
        for (int pos = 0 ; pos < MAX; pos++)  
            System.out.println(B[pos]);  
        cargar_variable_simple(a);  
        System.out.println("La variable es:" + a);  
    }  
    private static void cargar_variable_simple(int c) {  
        c = 10;  
    }  
    private static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos] = pos * 2;  
    }  
}
```

Memoria RAM - main

B = | 0 | 2 | 4 | 6 | 8 |
a = 20

Memoria RAM -
cargar_arreglo

arr = 0xf3a3fd4a
(recibe la dirección de
memoria)
pos = 0

Ejemplo

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] B=new int[MAX];  
        int a = 20;  
        cargar_arreglo(B);  
        System.out.println("Los datos son:");  
        for (int pos = 0 ; pos < MAX; pos++)  
            System.out.println(B[pos]);  
        cargar_variable_simple(a);  
        System.out.println("La variable es:"+a);  
    }  
    private static void cargar_variable_simple(int c) {  
        c = 10;  
    }  
    private static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
}
```

Memoria RAM - main

B = | 0 | 2 | 4 | 6 | 8 |
a = 20
pos = 0

Ejemplo

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] B=new int[MAX];  
        int a = 20;  
        cargar_arreglo(B);  
        System.out.println("Los datos son:");  
        for (int pos = 0 ; pos < MAX; pos++)  
            System.out.println(B[pos]);  
        cargar_variable_simple(a);  
        System.out.println("La variable es:"+a);  
    }  
    private static void cargar_variable_simple(int c) {  
        c = 10;  
    }  
    private static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
}
```

Memoria RAM - main

B = | 0 | 2 | 4 | 6 | 8 |
a = 20

Memoria RAM -
cargar_variable_simple

c=20

Ejemplo

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] B=new int[MAX];  
        int a = 20;  
        cargar_arreglo(B);  
        System.out.println("Los datos son:");  
        for (int pos = 0 ; pos < MAX; pos++)  
            System.out.println(B[pos]);  
        cargar_variable_simple(a);  
        System.out.println("La variable es:"+a);  
    }  
    private static void cargar_variable_simple(int c) {  
        c = 10;  
    }  
    private static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
}
```

Memoria RAM - main

B = | 0 | 2 | 4 | 6 | 8 |
a = 20

Memoria RAM -
cargar_variable_simple

c=~~20~~ 10

Ejemplo

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] B=new int[MAX];  
        int a = 20;  
        cargar_arreglo(B);  
        System.out.println("Los datos son:");  
        for (int pos = 0 ; pos < MAX; pos++)  
            System.out.println(B[pos]);  
        cargar_variable_simple(a);  
        System.out.println("La variable es:"+a);  
    }  
    private static void cargar_variable_simple(int c) {  
        c = 10;  
    }  
    private static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
}
```

Memoria RAM - main

B = | 0 | 2 | 4 | 6 | 8 |
a = 20

Cargar un arreglo desde teclado



//Hacer un programa que cargue en un arreglo de enteros 5 valores desde teclado y lo imprima.

```
public class Clase_5_Ejemplo_1{
    final static int MAX = 5;
    public static void main (String[] args) {
        int[] enteros=new int[MAX];
        cargarArreglo(enteros);
        mostrarArreglo(enteros);
    }
    public static void cargarArreglo(int[] arrenteros){
        for (int pos = 0; pos < MAX; pos++){
            System.out.println ("Ingrese el entero de la posición " + pos + ": ");
            arrenteros[pos] = Utils.leerInt();
        }
    }
    public static void mostrarArreglo(int[] arrenteros){
        for (int pos = 0; pos < MAX; pos++){
            System.out.println("arrenteros["+pos+"] -> "+arrenteros[pos]);
        }
    }
}
} // fin del class
```

Consideraciones en arreglos

Teniendo en cuenta el tipo de pasaje de parámetros en java, devolver un arreglo no estaría bien siempre y cuando esté trabajando sólo sobre el arreglo original.

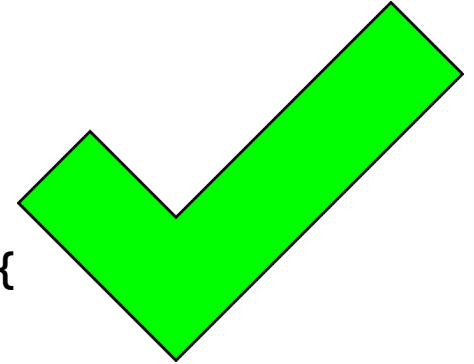
```
public class Clase_5_Ejemplo_3 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] A=new int[MAX], B=new int[MAX];  
        B=cargar_arreglo(A);  
    }  
    public static int[] cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
        }  
        return arr;  
    }  
}
```



Consideraciones en arreglos

El método debe ser void para que no retorne nada ya que se modifica el arreglo original pasado por parámetro.

```
public class Clase_5_Ejemplo_3 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] A=new int[MAX];  
        cargar_arreglo(A);  
    }  
    public static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
        }  
    }  
}
```

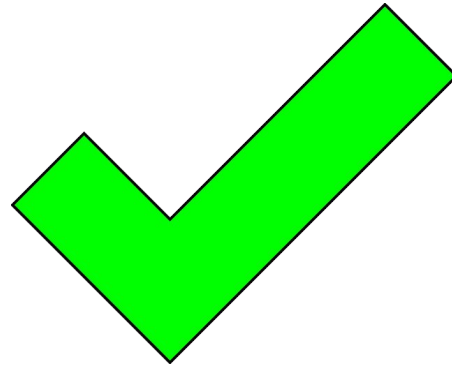


Consideraciones en arreglos



Si se quiere devolver un arreglo que es resultado de recorrer y procesar otro, podría ser una buena estrategia para resolver un cierto problema.

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] A=new int[MAX], B=new int[MAX];  
        cargar_arreglo(A);  
        B=procesar(A);  
    }  
    public static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
}  
public static int[] procesar(int[] arr) {  
    int[] resultado=new int[MAX];  
    for (int pos = 0 ; pos < MAX; pos++)  
        resultado[pos]=arr[pos]*arr[pos]+5;  
    return resultado;  
}
```



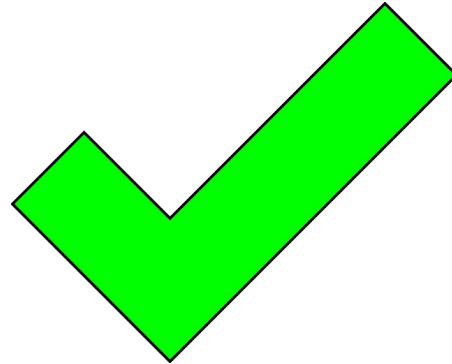
En este caso se necesitan conservar los dos: el original **A** y el resultado procesado **B**

Consideraciones en arreglos



Si se quiere devolver un arreglo que es resultado de recorrer y procesar otro, podría ser una buena estrategia pasarlo como parámetro al que quiero como resultado.

```
public class Clase_5_Ejemplo_4 {  
    final static int MAX = 5;  
    public static void main (String[] args) {  
        int[] A=new int[MAX], B=new int[MAX];  
        cargar_arreglo(A);  
        procesar(A,B);  
    }  
    public static void cargar_arreglo(int[] arr) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            arr[pos]=pos*2;  
    }  
    public static void procesar(int[] arr, int[] resultado) {  
        for (int pos = 0 ; pos < MAX; pos++)  
            resultado[pos]=arr[pos]*arr[pos]+5;  
    }  
}
```



En este caso le paso B como parámetro y lo modifico adentro por lo cual no hay que devolverlo.

Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:

12	3	6	23	3	8	9	7	4	6	← Valores
0	1	2	3	4	5	6	7	8	9	← Posiciones

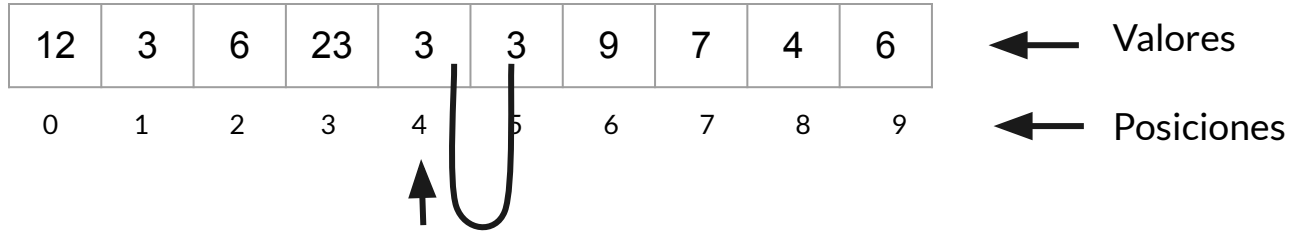
↑

posición 4, que no es la cuarta sino la quinta porque arrancamos de 0

Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:

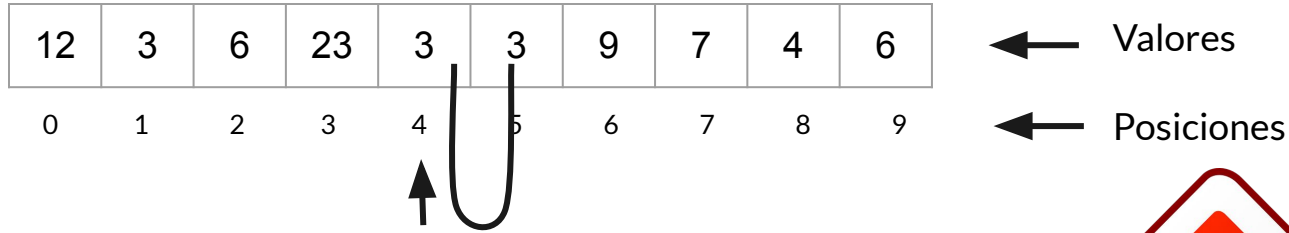


posición 4, que no es la cuarta sino la quinta porque arrancamos de 0

Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:



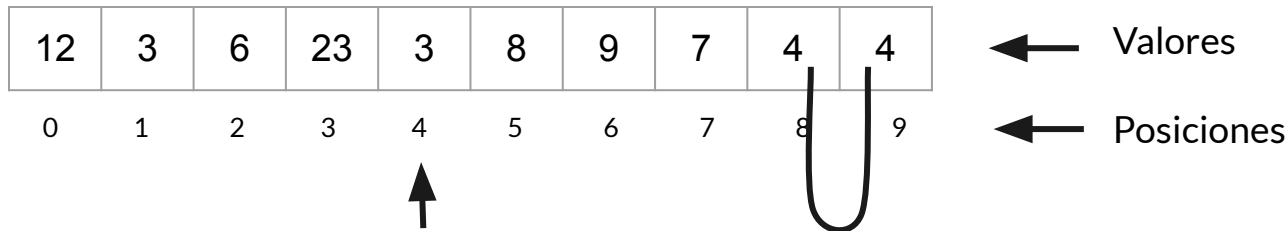
posición 4, que no es la cuarta sino la quinta porque arrancamos de 0



Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:



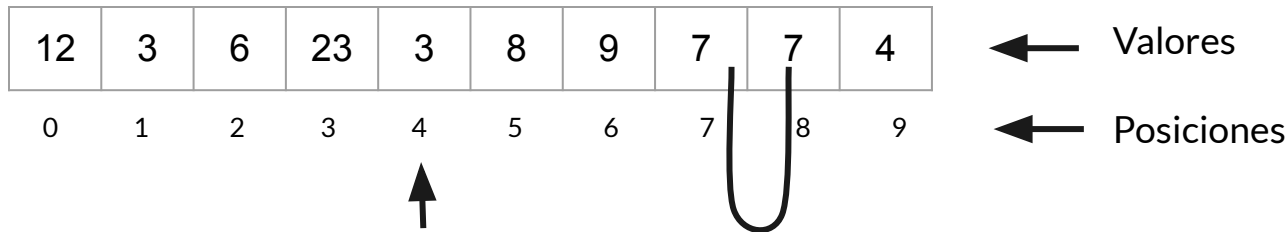
posición 4, que no es la cuarta sino la quinta porque arrancamos de 0

Importante: Se deberán desplazar un lugar a derecha todos los elementos que están a la derecha de la cuarta posición.

Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:



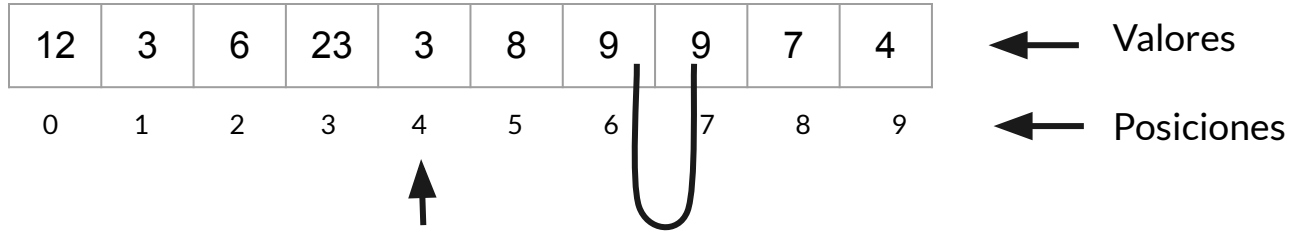
posición 4, que no es la cuarta sino la quinta porque arrancamos de 0

Importante: Se deberán desplazar un lugar a derecha todos los elementos que están a la derecha de la cuarta posición.

Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:



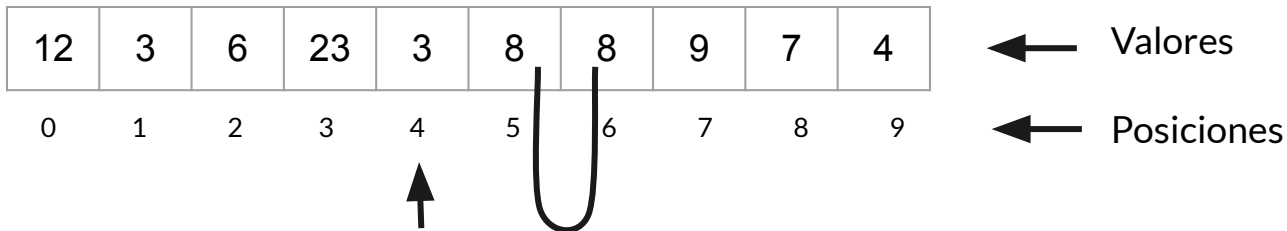
posición 4, que no es la cuarta sino la quinta porque arrancamos de 0

Importante: Se deberán desplazar un lugar a derecha todos los elementos que están a la derecha de la cuarta posición.

Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:



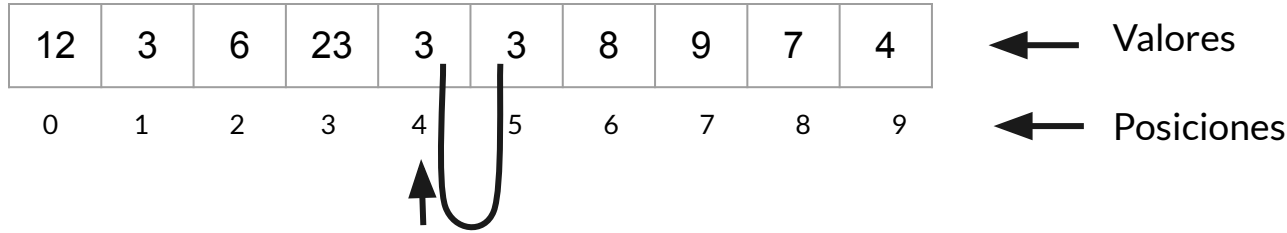
posición 4, que no es la cuarta sino la quinta porque arrancamos de 0

Importante: Se deberán desplazar un lugar a derecha todos los elementos que están a la derecha de la cuarta posición.

Corrimiento a derecha

Generalmente asociado al hecho de **insertar** un elemento en un arreglo (ordenado o no)

El usuario quiere insertar un elemento en la quinta posición, entonces:



posición 4, que no es la cuarta sino la quinta porque arrancamos de 0

- El elemento en la posición `pos`, queda copiado en la posición `pos + 1` y se pierde el último elemento.
- Si tuviera que insertar un elemento en esa posición sería algo como:
 - `arreglo[pos]=valor; // pos es la ingresada por el usuario`

Corrimiento a izquierda

Generalmente asociado al hecho de **eliminar** un elemento en un arreglo (ordenado o no)

El usuario quiere eliminar el elemento de la séptima posición, entonces:

12	3	6	23	3	8	9	7	4	6
0	1	2	3	4	5	6	7	8	9

← Valores

← Posiciones

↑

posición 6 porque arrancamos de 0

Importante: Se deberán desplazar un lugar a izquierda todos los elementos que están a la derecha de la séptima posición.

Corrimiento a izquierda

Generalmente asociado al hecho de **eliminar** un elemento en un arreglo (ordenado o no)

El usuario quiere eliminar el elemento de la séptima posición, entonces:

12	3	6	23	3	8	7	7	4	6
0	1	2	3	4	5	6	7	8	9

← Valores

← Posiciones

posición 6 porque arrancamos de 0

Importante: Se deberán desplazar un lugar a izquierda todos los elementos que están a la derecha de la séptima posición.

Corrimiento a izquierda

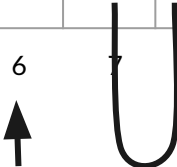
Generalmente asociado al hecho de **eliminar** un elemento en un arreglo (ordenado o no)

El usuario quiere eliminar el elemento de la séptima posición, entonces:

12	3	6	23	3	8	7	4	4	6
0	1	2	3	4	5	6	7	8	9

← Valores

← Posiciones



posición 6 porque arrancamos de 0

Importante: Se deberán desplazar un lugar a izquierda todos los elementos que están a la derecha de la séptima posición.

Corrimiento a izquierda

Generalmente asociado al hecho de **eliminar** un elemento en un arreglo (ordenado o no)

El usuario quiere eliminar el elemento de la séptima posición, entonces:

12	3	6	23	3	8	7	4	6	6
0	1	2	3	4	5	6	7	8	9

← Valores

← Posiciones

posición 6 porque arrancamos de 0

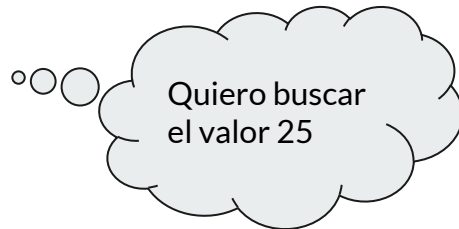
El último elemento queda duplicado.

Estructuras ordenadas

Se refiere a una propiedad que posee su composición respecto de sus elementos.

```
private static int buscar_pos_desordenado(int[] arr,int valor) {  
    int pos = 0;  
    while ( (pos<MAX) &&(arr[pos]!=valor)) {  
        pos++;  
    }  
    if (pos<MAX)  
        return pos;  
    else  
        return -1;  
}
```

12	3	6	23	3	8	9	7	4	6
----	---	---	----	---	---	---	---	---	---



```
private static int buscar_pos_ordenado(int[] arr,int valor){  
    int pos = 0;  
    while ( (pos<MAX) &&(arr[pos]>valor)) {  
        pos++;  
    }  
    if ((pos<MAX) &&(arr[pos]==valor))  
        return pos;  
    else  
        return -1;  
}
```

23	12	9	8	7	6	6	4	3	3
----	----	---	---	---	---	---	---	---	---



Métodos de ordenamiento

Los algoritmos de ordenamiento permiten ordenar sus elementos.
Los métodos más populares son:

- Selección
- Inserción
- Burbujeo

Selección

```
public static void seleccion(int arr[]) {
    int i, j, menor, pos, tmp;
    for (i = 0; i < MAX; i++) { // tomamos como menor el primero
        menor = arr[i]; // de los elementos que quedan por ordenar
        pos = i; // y guardamos su posición
        for (j = i + 1; j < MAX; j++){ // buscamos en el resto
            if (arr[j] < menor) { // del array algún elemento
                menor = arr[j]; // menor que el actual
                pos = j;
            }
        }
        if (pos != i){ // si hay alguno menor se intercambia
            tmp = arr[i];
            arr[i] = arr[pos];
            arr[pos] = tmp;
        }
    }
}
```

Inserción



```
public static void insercion(int arr[]){
    for (int i = 1; i < MAX; i++) {
        int aux = arr[i];
        int j = i - 1;
        while ((j >= 0) && (arr[j] > aux)){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = aux;
    }
}
```

Burbujeo



```
public static void burbujeo(int[] arr){
    int temp;
    for(int i = 1; i < MAX; i++){
        for (int j = 0 ; j < MAX - 1; j++){
            if (arr[j] > arr[j+1]){
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```


Métodos para la carga de arreglos de forma aleatoria

```
import java.util.Random;

public class Clase_5_Cargar_Aleatorio {
    public static final int MAX = 10;
    public static final int MAXVALOR = 10;
    public static final int MINVALOR = 1;
    public static void main(String[] args) {
        char [] arrchar = new char[MAX];
        int [] arrint = new int[MAX];
        double [] arrdouble = new double[MAX];
        cargar_arreglo_aleatorio_char(arrchar);
        cargar_arreglo_aleatorio_int(arrint);
        cargar_arreglo_aleatorio_double(arrdouble);
        imprimir_arreglo_char(arrchar);
        imprimir_arreglo_int(arrint);
        imprimir_arreglo_double(arrdouble);
    }
    //carga de arreglo de char con valores de 'a' a la 'z'
    public static void cargar_arreglo_aleatorio_char(char [] arr){
        Random r = new Random();
        for (int pos = 0; pos < MAX; pos++){
            arr[pos]=(char) (r.nextInt(26) + 'a');
        }
    }
    //carga de arreglo de int con valores de MINVALOR a MAXVALOR
    public static void cargar_arreglo_aleatorio_int(int [] arr){
        Random r = new Random();
        for (int pos = 0; pos < MAX; pos++){
            arr[pos]=(r.nextInt(MAXVALOR-MINVALOR+1) + MINVALOR);
        }
    }
}
```

```
//carga de arreglo de double con valores de MINVALOR a MAXVALOR
public static void cargar_arreglo_aleatorio_double(double [] arr){
    Random r = new Random();
    for (int pos = 0; pos < MAX; pos++){
        arr[pos]=((MAXVALOR-MINVALOR+1)*r.nextDouble() +
        MINVALOR*1.0);
    }
}
//impresion de arreglo de char
public static void imprimir_arreglo_char(char [] arr){
    for (int pos = 0; pos < MAX; pos++){
        System.out.println("nombre_arreglo["+pos+"]=>: "+arr[pos]);
    }
}
//impresion de arreglo de int
public static void imprimir_arreglo_int(int [] arr){
    for (int pos = 0; pos < MAX; pos++){
        System.out.println("nombre_arreglo["+pos+"]=>: "+arr[pos]);
    }
}
//impresion de arreglo de double
public static void imprimir_arreglo_double(double [] arr){
    for (int pos = 0; pos < MAX; pos++){
        System.out.println("nombre_arreglo["+pos+"]=>: "+arr[pos]);
    }
}
}
```

Buscar un elemento en un arreglo ordenado creciente

//Hacer un programa que dado un arreglo de enteros ordenado creciente de tamaño 10, encuentre la posición donde se halla
//un número entero dado. Si existe, muestre esa posición por pantalla, o indique que no existe.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
public class Clase_5_Buscar {
    public final static int MAX = 10;
    public static void main (String [] args) {
        int [] arrenteros = new int [MAX];
        cargarArreglo(arrenteros); // se supone implementado el metodo cargarArreglo
        int pos,numero;
        numero=45; // es para el ejemplo pero se puede pedir por consola por ejemplo
        pos = buscar_pos_arreglo_ord_crec(arrenteros,numero);
        if ((pos<MAX)&&(arrenteros[pos]==numero)){
            System.out.println("La posición de "+numero+" es: "+pos);
        }
        else{
            System.out.println(numero + " no existe en el arreglo");
        }
    }
    //La posición que retorna no necesariamente significa que esté ahí, es donde debería estar
    public static int buscar_pos_arreglo_ord_crec(int[] arr,int numero) {
        int pos = 0;
        while ((pos<MAX)&&(arr[pos]<numero)){
            pos++;
        }
        return pos;
    }
}
```

Cargar un arreglo desde teclado



// Hacer un programa que cargue en un arreglo de enteros naturales 5 valores desde teclado, los inserte ordenados ascendentemente y lo imprima.